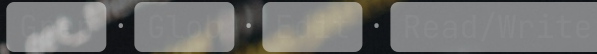


Built-in Tools

เลือกเครื่องมือให้ถูกงาน —



Domain 2: Task 2.5 — Claude Certified Architect

Built-in Tools คืออะไร

- ชุดเครื่องมือมาตรฐานที่ **Claude Code** มีให้ใช้สำรวจและแก้ไข codebase — หลัก ๆ คือ `Grep` · `Glob` · `Edit` · `Read/Write`
- สิ่งที่ Task นี้ทดสอบ **ไม่ใช่** ว่าแต่ละเครื่องมือ "ทำงานอย่างไร"
- แต่คือ "**เลือกตัวไหนกับงานแบบไหน**" ให้ประหยัด context และแม่นยำที่สุด

การ**เลือกผิดเครื่องมือ** คือกับดักหลักที่ข้อสอบวางไว้

Grep vs Glob

★ แคนกลางของ Task

Grep

ค้นหา "**เนื้อหาข้างในไฟล์**" (file CONTENTS) ตาม pattern

ผู้เรียกฟังก์ชัน · ข้อความ error · `import` statement · การกำหนดค่าตัวแปร

Glob

จับคู่ "**path ของไฟล์**" (file PATHS) ตาม pattern ของชื่อ

หาไฟล์ตามชื่อ · นามสกุล · โครงสร้างโฟลเดอร์

Key Concept

"Grep finds what is **INSIDE** files. Glob finds files by their **NAMES**."

เกือบทุกกับดักของ Task นี้คือการสลับสองตัวนี้

ตัวอย่างการใช้ Grep กับ Glob

Grep (เนื้อหา)

"processLegacyOrder" – หาที่เรียกฟังก์ชัน

"timeout" – หาข้อความ error

"import.*from 'utils/auth'" – หา import

Glob (ชื่อไฟล์)

**/*.test.tsx – หาไฟล์ test

**/config.* – หาไฟล์ config

content/domains/**/*.mdx – MDX ตามตำแหน่ง

หลักวิธีย่อย: สิ่งที่เราตามหา "อยู่ข้างในไฟล์" → **Grep** · เป็น "ตัวไฟล์ตามชื่อ/นามสกุล/ที่อยู่" → **Glob**

เครื่องมือแก้ไขไฟล์: Edit

✓ Edit

แทนที่ข้อความแบบเจาะจงด้วยการ match ข้อความที่ **ไม่ซ้ำ (unique text)** โดยไม่ต้องโหลดทั้งไฟล์ — เร็วและแม่นยำ

ตัวอย่าง

```
old_string: "function processOrder(id: string)"  
new_string: "function processOrder(id: string, validate: boolean = true)"
```

⚡ ข้อจำกัด

Edit จะ**ล้มเหลว**เมื่อข้อความเป้าหมายปรากฏหลายที่ (ไม่ unique)

วิธีแก้: ขยาย `old_string` ให้รวม context สอนข้างเพื่อให้ไม่ซ้ำ หรือตั้ง `replace_all: true`

Read + Write และลำดับการ escalate

Read + Write

โหลดทั้งไฟล์เข้ามาแก้ — ใช้เป็น "**ทางเลือกสุดท้าย**" (last resort) เท่านั้น

หลังจาก Edit ล้มเหลวทั้งที่ขยาย anchor และลอง `replace_all` แล้ว

ลำดับ escalate เมื่อ Edit เจอ match ชั่ว

ขยาย `old_string` → `replace_all: true` → Read + Write

 กระโดดไป Read + Write **ทันที** ที่ Edit พลาดครั้งแรก คือกับดักที่ข้อสอบชอบวาง

ลำดับการสำรวจ codebase ที่ถูกต้อง

× Anti-pattern

"อ่านทุกไฟล์ล่วงหน้าก่อนจะรู้ว่าอะไรเกี่ยวข้อง" (Reading all files upfront) — เปลือง context tokens โดยเปล่าประโยชน์

1 **Grep** หา entry point (ชื่อฟังก์ชัน/คลาส หรือข้อความ error)

2 **Read** เฉพาะไฟล์ที่เกี่ยวข้อง เพื่อไล่ import และเข้าใจโครงสร้าง

3 **Grep** ซ้ำ เพื่อหาการใช้ wrapper function

4 **Read** เฉพาะไฟล์ที่มีเหตุผลรองรับจากสิ่งที่เพิ่งค้นเจอ

Pattern จัดการฟังก์ชันที่ถูก deprecated

★ ออกสอบบ่อย

"**Grep**, then **Glob**, then **Grep again**"

- 1 **Grep** ชื่อฟังก์ชัน — เจอทุกไฟล์ที่อ้างถึงโดยตรง
- 2 **Glob** หาไฟล์ test ที่เป็น sibling — ตามธรรมเนียมการตั้งชื่อ (`OrderProcessor.ts` → `OrderProcessor.test.tsx`)
- 3 **Grep** ชื่อ wrapper — จับ coverage ทางอ้อมเมื่อฟังก์ชันถูกห่อหรือ re-export

! Exam Traps ที่ต้องระวัง

1. ใช้ **Glob** หาผู้เรียกฟังก์ชัน — ผิด เพราะ Glob จับแค่ path ไม่ดูเนื้อหา
2. ใช้ **Grep** หา pattern นามสกุล/ชื่อไฟล์ — ผิด เพราะงานนี้เป็นของ Glob
3. อ่านทุกไฟล์ล่วงหน้า — เปลือง context budget โดยไม่จำเป็น
4. ใช้ **Read + Write** เป็นค่าเริ่มต้นในการแก้ไข — ผิด เพราะ Edit เร็วและแม่นยำกว่า
5. escalate ไป Read + Write **ทันที** ที่ Edit ล้มเหลว — ผิด เพราะขยาย anchor หรือ `replace_all` แก้ได้ส่วนใหญ่



Practice Scenario

โจทย์

ต้องการหา "ทุกไฟล์ที่เรียก `processLegacyOrder()` " พร้อมทั้ง "ไฟล์ test ของไฟล์ผู้เรียกเหล่านั้น" — ควรใช้วิธีไหน?

- A. Glob หา `**/*processLegacyOrder*` เพื่อหาไฟล์ผู้เรียก แล้ว Glob หาไฟล์ test อีกครั้ง
- B. Read ทุกไฟล์ใน repo แล้วไล่อ่านหาการเรียก `processLegacyOrder` ทีละไฟล์
- C. Grep หา `processLegacyOrder` เพื่อเจอผู้เรียก (ฟังก์ test ที่ import ตรง ๆ) แล้ว Glob หาไฟล์ test ที่เป็น sibling ของแต่ละผู้เรียก (เช่น `**/OrderProcessor.test.*`) เพื่อจับ test ที่เรียกผ่าน source module โดยไม่เอ่ยชื่อฟังก์ชัน
- D. Grep หา `processLegacyOrder` แล้ว Grep หา `*.test.*` เพื่อหาไฟล์ test



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option C

Grep หา `processLegacyOrder` เพื่อเจอผู้เรียก (ฟังก์ชัน test ที่ import ตรง ๆ) แล้ว Glob หาไฟล์ test ที่เป็น sibling ของแต่ละผู้เรียก เพื่อครอบคลุม test ที่เรียกผ่าน module โดยไม่เอ่ยชื่อฟังก์ชัน

→ ใช้ทั้ง **Grep (เนื้อหา)** และ **Glob (ชื่อไฟล์)** ให้ถูกงาน

A ผิด — Glob หาผู้เรียก แต่ Glob จับแค่ path ไม่เห็นการเรียกในเนื้อไฟล์

B ผิด — อ่านทุกไฟล์ล่องหน้า เปลี่ยน context อย่างมหาศาล

D ผิด — ใช้ Grep หา pattern ชื่อไฟล์ test ทั้งที่เป็นงานของ Glob และยังพลาด test ที่ไม่ได้ตั้งชื่อตรงกับฟังก์ชัน



Build Exercise: อัปเดตฟังก์ชันที่ deprecated

- 1 **Grep** หาฟังก์ชันที่ deprecated เพื่อระบุไฟล์ผู้เรียก
- 2 **Glob** หาไฟล์ test ที่ match กับ pattern ของผู้เรียก
- 3 **Read** ไฟล์ผู้เรียกแต่ละไฟล์แบบค่อยเป็นค่อยไป (incrementally)
- 4 **Edit** เพื่อแทนที่การเรียกแบบเก่าด้วย API ใหม่
- 5 เมื่อเจอ match ไม่ unique: ขยาย `old_string` หรือ `replace_all: true` — escalate ไป Read + Write เฉพาะตอนแยกความกำกวมไม่ได้จริง ๆ



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนาต้องการหา "ทุกไฟล์ที่เรียกฟังก์ชันที่ deprecated `processLegacyOrder()` " และหา "ไฟล์ test ของไฟล์ผู้เรียกเหล่านั้น" ด้วย — ลำดับการใช้ tool แบบไหนที่ถูกต้อง?

- A. Glob หา `*processLegacyOrder*` เพื่อเจอผู้เรียก แล้ว Grep หาไฟล์ test
- B. Read ทุก source file เพื่อไล่หาฟังก์ชันด้วยตนเอง แล้ว Read ทุกไฟล์ test
- C. Grep หา `processLegacyOrder` เพื่อเจอผู้เรียก แล้ว Glob หาไฟล์ test ตาม pattern ชื่อไฟล์ผู้เรียก (เช่น `**/*.test.tsx`)
- D. Bash ด้วย `find` และ `xargs grep` สำหรับทั้งสองขั้นตอน

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option C

Grep ค้นหาไฟล์ — เหมาะกับการหาผู้เรียกฟังก์ชัน ส่วน Glob จับคู่ path ของไฟล์ — เหมาะกับการหาไฟล์ test ตาม pattern ชื่อ นี่คือลำดับสองขั้นที่ดีที่สุด: ค้นหาไฟล์ก่อน แล้วจับคู่ชื่อไฟล์

ทำไมข้ออื่นผิด

A — Glob จับแค่ path ไม่ใช่เนื้อหา หาผู้เรียกฟังก์ชันไม่ได้ และสลับหน้าที่ของสอง tool

B — อ่านทุกไฟล์ล่วงหน้าเป็นตัวอย่าง context budget เปลือง token กับไฟล์ที่ไม่เกี่ยวข้อง คือ anti-pattern ที่ข้อสอบหักคะแนน

D — แม้ทำงานได้จริง แต่เสี่ยง built-in tool ที่ออกแบบมาเพื่องานนี้ ข้อสอบต้องการให้เลือก built-in tool ที่เหมาะกับแต่ละงาน



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนาลองใช้ Edit เพื่อแทนที่การเรียกฟังก์ชัน แต่ Edit ล้มเหลวเพราะข้อความปรากฏ 3 ที่ในไฟล์ (ไม่ unique) — ขั้นตอนถัดไปที่ต้องทำอะไร?

- A. ใช้ Edit ด้วย `old_string` ที่กว้างขึ้น รวม context รอบข้างเพื่อให้ match ไม่ซ้ำ
- B. ใช้ Bash กับ `sed` เพื่อแทนที่ทุกจุดพร้อมกัน
- C. ถ้า context เพิ่มแล้วยังไม่ unique ให้ fallback เป็น Read โหลดทั้งไฟล์ แล้ว Write ออกมาเวอร์ชันที่แก้แล้วทั้งไฟล์
- D. เติม comment ลงไฟล์เพื่อให้จุดเป้าหมาย unique, Edit บรรทัดนั้น แล้วค่อยลบ comment

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option A (แล้ว fallback เป็น Option C)

ลองใหม่ด้วย Edit ก่อนโดยขยาย `old_string` ให้ยาวและเจาะจงขึ้น มักทำให้ match unique ได้ — ถ้าเพิ่ม context แล้วยังไม่ unique ค่อย fallback เป็น Option C (Read + Write) ซึ่งเชื่อถือได้แต่กิน context token มากกว่า

ทำไมข้ออื่นผิด

- B — Bash กับ `sed` เลี่ยง built-in Edit และขาดการตรวจ unique-match อาจแก้จุดโดยไม่ตั้งใจ
- D — เพิ่ม comment ชั่วคราวเปราะบางและเสียงพลาต สร้างการแก้ไขที่ไม่จำเป็นและอาจหลงเหลือ artefact
- C — ไม่ผิด แต่เป็น fallback หลังจากลอง A แล้วยังไม่ unique ไม่ควรกระโดดมาใช้ทันที



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนาต้องการหาไฟล์ test ที่เป็น TypeScript ทั้งหมดในโปรเจกต์ — ควรใช้ tool และ pattern แบบไหน?

A. Grep หา `"test"` เพื่อหาไฟล์ที่มีโค้ดเกี่ยวกับ test

B. Glob หา `**/*.test.tsx` เพื่อหาไฟล์ test ตามธรรมเนียมการตั้งชื่อ

C. Read ไฟลเดอร์ root เพื่อ list ทุกไฟล์ แล้วกรองด้วยตนเอง

D. Bash ด้วย `find . -name "*.test.tsx"` เพื่อค้นไฟล์ test

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

Glob จับคู่ path ไฟล์ตาม pattern ชื่อ การหาไฟล์ test ตามนามสกุล (`**/*.test.tsx`) คือสิ่งที่ Glob ออกแบบมา โดยเฉพาะ และคืน path ที่ match เรียงตามเวลาแก้ไขล่าสุด

ทำไมข้ออื่นผิด

- A — Grep ค้นหาไฟล์ ไม่ใช่ path เป็น tool ผิดงาน และจะได้ไฟล์ที่ไม่ใช่ test แต่บังเอิญมีคำว่า "test" ในโค้ดติดมาด้วย
- C — อ่านไฟล์เตอร์แล้วกรองเองเป็นงาน manual ไม่กรองตาม pattern ด้วยประสิทธิภาพเทียบกับ Glob
- D — แม้ `find` ใช้ได้ แต่ข้อสอบต้องการให้ใช้ built-in Glob ซึ่งเป็น tool เฉพาะสำหรับค้นไฟล์ตาม path



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนาต้องการเข้าใจการทำงานของ module ที่ซับซ้อน จึงเริ่มด้วยการ Read ทุกไฟล์ทั้ง 47 ไฟล์ใน module นั้น — อะไรคือสิ่งที่ผิดกับวิธีนี้?

A. ไม่มีอะไรผิด — อ่านทุกไฟล์ให้ความเข้าใจที่ครบถ้วนที่สุด

B. การอ่านทุกไฟล์ล่วงหน้าเป็นตัวทำลาย context budget วิธีที่ถูกคือทำแบบ incremental: Grep หา entry point ก่อน แล้ว Read เฉพาะไฟล์ที่ Grep ระบุว่าเกี่ยวข้อง

C. ควรใช้ Glob ก่อนเพื่อกรองว่าจะอ่านไฟล์ไหน

D. ควรขอเอกสารจากทีมแทนที่จะอ่านโค้ด

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

การโหลดทุกไฟล์เข้ามาก่อนจะรู้ว่าอะไรเกี่ยวข้องกับ context window ไปกับไฟล์ที่ส่วนใหญ่ไม่เกี่ยวข้อง วิธีที่ถูกคือทำแบบ incremental: Grep หา entry point (ชื่อฟังก์ชัน/คลาส หรือข้อความ error) แล้ว Read เพื่อไล่ import และ trace flow จาก entry point เหล่านั้น

ทำไมข้ออื่นผิด

- A — อ่าน 47 ไฟล์เปลือง context กับไฟล์ที่ส่วนใหญ่ไม่เกี่ยวข้อง เป็น anti-pattern ที่ใหญ่ที่สุดในการสำรวจ codebase
- C — Glob หาไฟล์ตาม pattern ชื่อ ไม่ได้บอกว่าไฟล์ไหนเกี่ยวข้องกับงานที่ทำ การกรองตามนามสกุลไม่ได้ชี้ความเกี่ยวข้อง
- D — เอกสารอาจไม่มีหรือล้าสมัย ข้อสอบทดสอบกลยุทธ์การสำรวจโค้ด ไม่ใช่แนวปฏิบัติเรื่องเอกสาร



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

ฟังก์ชัน `processOrder` ถูกนิยามใน `orders.ts` , re-export ผ่าน `utils/index.ts` แบบ barrel export และถูกใช้โดย 5 ไฟล์ที่ import จาก `utils` — Grep แบบง่ายหา `"processOrder"` เจอตัวนิยามและ barrel file แต่พลาดผู้ใช้ไป 3 ใน 5 ไฟล์ เพราะอะไร?

- A. Grep มีลิมิตจำนวนไฟล์ และหยุดหลังเจอ match ถึงจำนวนหนึ่ง
- B. ผู้ใช้ที่หายไป 3 ไฟล์ import จาก `"utils"` โดยไม่เอ่ยชื่อ `processOrder` ตรง ๆ — ใช้ namespace import (เช่น `import * as utils`) แล้วเรียก `utils.processOrder` ซึ่ง Grep แบบง่ายจับไม่ได้
- C. Grep pattern ต้องใช้ regex เพื่อ match ชื่อฟังก์ชัน
- D. ผู้ใช้อยู่คนละ branch ของ repository

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

ผู้ใช้ที่ใช้ namespace import (`import * as utils from "utils"`) เรียกฟังก์ชันในรูปแบบ `utils.processOrder` ซึ่ง Grep หา `"processOrder"` ตรง ๆ จับไม่เจอ วิธีที่ถูกคือทำหลายขั้น: Grep หาตัวนิยาม, Read เพื่อระบุชื่อที่ export และ barrel pattern ทั้งหมด แล้ว Grep แต่ละ pattern รวมถึงชื่อ barrel module ด้วย

ทำไมข้ออื่นผิด

A — Grep ไม่มี default file limit มันค้นทุกไฟล์ใน scope ที่ระบุ

C — การ match string ตรง ๆ กับ `"processOrder"` เพียงพอสำหรับการอ้างอิงถึงโดยตรง ปัญหาไม่ใช่ regex แต่เป็นการอ้างทางอ้อมผ่าน namespace import

D — Grep ค้นใน working tree ปัจจุบัน ไฟล์ใน branch อื่นไม่เกี่ยวกับการสืบสวนนี้